

FORMATION EMBARQUÉE

Antiséche — Arduino Stm32

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Antisèche — Arduino & STM32

Arduino — l'API en une page

```
void setup() { }           // une fois au démarrage
void loop() { }           // en boucle, indéfiniment

pinMode(p, OUTPUT | INPUT | INPUT_PULLUP);
digitalWrite(p, HIGH|LOW);  bool v = digitalRead(p);  // p appuyé = LOW en pull-up
int a = analogRead(A0);    // 0..1023 (Uno) · 0..4095 (ESP32)
analogWrite(p, 0..255);    // PWM (broches ~ seulement)

uint32_t t = millis();     // ms depuis le boot (déborde à 49,7 j)
uint32_t u = micros();     // µs (déborde à ~70 min)

Serial.begin(115200); Serial.print(x); Serial.println(F("en flash"));
if (Serial.available()) c = Serial.read();

attachInterrupt(digitalPinToInterrupt(2), isr, RISING|FALLING|CHANGE);
noInterrupts(); /* section critique */ interrupts();
map(v, 0,1023, 0,100); constrain(v, min, max);
```

Brochage Uno (l'essentiel)

Fonction	Broches
PWM	3, 5, 6, 9, 10, 11 (~)
Interruptions externes	2, 3
I2C	A4 = SDA, A5 = SCL
SPI	10 CS · 11 MOSI · 12 MISO · 13 SCK
UART (occupé par l'USB)	0 RX, 1 TX
Analogique	A0-A5 (10 bits)

⚠ 20 mA max par broche · LED = **résistance 220 Ω** · moteur = **driver** · alim séparée pour servos/moteurs (masses communes) · RAM = 2 Ko → **F()** partout, pas de **String**.

Le motif non bloquant (à recopier partout)

```
uint32_t dernier = 0;
void loop() {
  uint32_t now = millis();
  if (now - dernier >= PERIODE) { dernier = now; /* tâche */ }
  // autres tâches à d'autres périodes, aucune ne bloque
}
```

Anti-rebond

```
bool brut = (digitalRead(BTN) == LOW);
if (brut != brut_prec) { t_chgt = now; brut_prec = brut; }
if (now - t_chgt > 20 && brut != stable) { stable = brut; /* front */ }
```

STM32 — HAL en une page

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET|GPIO_PIN_RESET);
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
uint32_t t = HAL_GetTick();          // ms - le millis() du STM32

HAL_UART_Transmit(&huart2, buf, len, 100);          // bloquant
HAL_UART_Receive_IT(&huart2, &octet, 1);           // IT - À RÉARMER !
HAL_UART_Transmit_DMA(&huart2, buf, len);           // DMA

HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, ccr);
__HAL_TIM_SET_AUTORELOAD(&htim2, arr);
HAL_TIM_Base_Start_IT(&htim3);

HAL_ADC_Start_DMA(&hadc1, (uint32_t*)tab, N);        // ADC circulaire
HAL_I2C_Mem_Read(&hi2c1, adr7 << 1, reg, 1, buf, n, 100); // << 1 !
HAL_SPI_TransmitReceive(&hspi1, tx, rx, n, 100);
```

Callbacks (à redéfinir, appelés depuis l'ISR)

```
void HAL_GPIO_EXTI_Callback(uint16_t pin);
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *h);
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h); // + réarmer !
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *h);
```

Les formules de timer (à savoir refaire)

```
f_PWM = f_timer / ((PSC+1) × (ARR+1))
rapport cyclique = CCR / (ARR+1)
période = (PSC+1) × (ARR+1) / f_timer
```

Objectif (f_timer = 100 MHz)	PSC	ARR
1 kHz PWM	99	999
20 kHz PWM	4	999
IT toutes les 1 ms	99	999
Compteur en µs (1 tick = 1 µs)	99	65535

Les 3 modes d'E/S

Mode	Suffixe	CPU
Bloquant	(aucun)	occupé à attendre
Interruption	<code>_IT</code>	libre, ISR à la fin
DMA	<code>_DMA</code>	libre, transfert autonome

Accès registres (sans HAL)

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN; // 1) TOUJOURS l'horloge d'abord
GPIOA->MODER &= ~GPIO_MODER_MODER5; // 2) nettoyer les bits
GPIOA->MODER |= GPIO_MODER_MODER5_0; // 3) écrire (01 = sortie)
GPIOA->ODR ^= GPIO_ODR_OD5; // 4) agir
```

Réflexes de débannage

Symptôme	Cause n°1
Périphérique muet	horloge RCC non activée
1 seul octet reçu en UART	<code>Receive_IT</code> non réarmé dans le callback
Valeur figée vue du main	<code>volatile</code> manquant
HardFault	pointeur nul/invalid → Fault Analyzer , regarder <code>BFAR</code> / <code>CFSR</code>
Plantage avec FreeRTOS	base de temps HAL sur SysTick, ou priorité NVIC trop haute
Comportement erratique	pile de tâche trop petite (<code>uxTaskGetStackHighWaterMark</code>)

FreeRTOS (CMSIS-v2) minimal

```
osThreadNew(TacheA, NULL, &attrA);
osDelay(100); // rend le CPU
osMessageQueuePut(q, &msg, 0, 0);
osMessageQueueGet(q, &msg, NULL, osWaitForever);
osMutexAcquire(m, osWaitForever); ... osMutexRelease(m);
// depuis une ISR : uniquement les variantes ...FromISR
```